MICROSOFT AGENT FRAMEWORK · RC

Building a Production AI Tutor

with Enterprise Observability

RAG · Q&A Evaluation · OpenTelemetry · GitHub Models · Foundry Local

Presenter:
Shreyan J D Fernandes
MVP-AI Services

For Technical Decision Makers & Developers

THE CHALLENGE

Building GenAI Apps That Are Production-Ready

Black-Box AI

Most agent frameworks give you a 200 OK — but zero insight into what the LLM actually did, which tools ran, or why the answer drifted.

No Cost Governance

Token consumption, latency spikes, and hallucination rates go undetected until they become expensive production incidents.

Proof-of-Concept Gap

POC demos look great. Production deployments fail quietly. Moving from demo to enterprise requires full lifecycle visibility.

A "200 OK" response is not a successful AI interaction. You need full lifecycle visibility.

THE SOLUTION

AI Tutor — Full-Stack Architecture

FRONTEND	Single-page app · RAG tab · Q&A Evaluator · Live Observability Dashboard
FASTAPI	REST backend · Upload / Query / Evaluate · Provider hot-switch endpoint
MAF AGENTS	RagTutorAgent · EvaluatorAgent · @tool search_documents · @tool fetch_rubric
LLM PROVIDERS	GitHub Models · Foundry Local (on-device) · Microsoft Foundry (Cloud)· OpenAI
OTEL	invoke_agent → chat → execute_tool · OTLP export · VS Code AI Toolkit

CORE TECHNOLOGY

Microsoft Agent Framework

```
# Correct MAF imports


from agent_framework.openai import \
    OpenAIChatClient
from agent_framework.azure import \
    AzureOpenAIChatClient
from agent_framework import Agent, tool

agent = Agent(
  client=OpenAIChatClient(
    api_key=token,
    model_id=model
  ),
  instructions=
"You are a tutor...",
  tools=
[search_documents]
)
```

Multi-Provider Support

GitHub Models, Foundry Local, Azure OpenAI, OpenAI — one factory function, four native connectors

@tool Decorator

Decorate any Python function as a tool. MAF handles invocation, JSON schema, and OTel spans automatically

Built-in Observability

Every agent call emits invoke_agent → chat → execute_tool spans following GenAI semantic conventions

FLEXIBILITY

Four Native MAF Providers — One Codebase

GitHub Models

FREE · DEV-FRIENDLY

`agent_framework.openai`

```
OpenAIChatClient(
  api_key=GITHUB_TOKEN,
  base_url=models.github.ai)
```

- No Azure subscription needed
- Free tier for developers
- GPT-4o-mini, Llama 3, Mistral

Foundry Local

ON-DEVICE · NO CLOUD

`agent_framework.openai`

```
OpenAIChatClient(
  api_key='local',
base_url=localhost:5272/v1)
```

- 100% offline inference
- Phi-3.5, Llama 3.2, Mistral
- OpenAI-compatible endpoint

Microsoft Foundry (Cloud)

ENTERPRISE · MANAGED IDENTITY

`agent_framework.azure`

```
AzureOpenAIChatClient(
  endpoint=AZURE_ENDPOINT,
  deployment_name=...)
```

- Keyless via DefaultAzureCredential
- Enterprise compliance & SLA
- Private network deployment

OpenAI

DEFAULT · FALLBACK

`agent_framework.openai`

```
OpenAIChatClient(
  api_key=OPENAI_API_KEY,
  model_id=gpt-4o-mini)
```

- Simple API key auth
- GPT-4o, GPT-4o-mini
- Drop-in default provider

Runtime hot-switch via POST /api/provider/switch — change providers mid-session without server restart

KNOWLEDGE BASE

RAG Pipeline — From Document to Grounded Answer

1 Ingest

Upload .txt / .pdf
Chunk → Embed
Upsert ChromaDB

`rag.ingest span`

2 Retrieve

Query embedding
Cosine similarity
Top-4 chunks

`rag.retrieve span`

3 Augment

Inject chunks
into agent prompt
With source cites

`chat span (MAF)`

4 Generate

Grounded answer
Cited sources
No hallucinations

`invoke_agent span`

Vector Store: ChromaDB (in-memory) · **Embedding:** SentenceTransformers · **Search:** Cosine similarity

EVALUATION ENGINE

EvaluatorAgent — Structured Learner Feedback

@tool fetch_rubric — Scoring Criteria

Accuracy
0–4 pts

Completeness
0–3 pts

Clarity
0–2 pts

Examples
0–1 pt

Total: /10 · Grade: A≥9 · B≥7 · C≥5 · D≥3 · F<3

EvaluatorAgent JSON Response


```
{
  "score": 8,
  "grade": "B",
  "strengths": [

"Accurate explanation of gradient descent"
  ],
  "improvements": [

"Add concrete learning rate examples"
  ],
  "detailed_feedback": "Good coverage of SGD
variants...",
  "hint": "Consider the role of momentum"
}
```

OBSERVABILITY

OpenTelemetry — Full Lifecycle Tracing

Trace Tree — RagTutorAgent Query

ai_tutor.rag_query	CUSTOM	1,247ms
invoke_agent RagTutorAgent	MAF	1,201ms
chat gpt-4o-mini	MAF	312ms
execute_tool search_documents	MAF	45m
rag.retrieve	CUSTOM	18
chat gpt-4o-mini (synthesis)	MAF	834ms

Export Targets

Console

Always on · dev default

VS Code AI Toolkit

OTLP:4318 · visual traces

Azure Monitor

Application Insights · prod

OTEL (VS Code AI Toolkit/Microsoft Foundry)

Any OTLP-compatible backend

BACKEND

FastAPI — REST API Reference

GET `/health`
Liveness probe + telemetry config + active provider

POST `/api/rag/upload`
Upload .txt / .pdf → chunk → embed → ChromaDB

POST `/api/rag/query`
RAG question answering via RagTutorAgent

GET `/api/rag/documents`
List all ingested source documents

DELETE `/api/rag/clear`
Wipe the entire ChromaDB vector store

POST `/api/eval/submit`
Evaluate learner answer via EvaluatorAgent

GET `/api/provider`
Active provider info + available providers list

POST `/api/provider/switch`
Hot-switch model provider at runtime

GET `/api/telemetry/traces`
Last N trace events from ring buffer

GET `/api/telemetry/metrics`
Runtime metric snapshot (queries, latency, errors)

GOVERNANCE

Enterprise Guardrails & Production Readiness

Source Grounding

- Answers cite document + chunk score
- RAG prevents parametric hallucination
- Empty store → explicit error, not fabrication

Full Observability

- Every LLM call traced end-to-end
- Token usage per span (GenAI conventions)
- Latency histogram per agent type

Performance Monitoring

- avg_rag_latency_ms tracked live
- avg_eval_latency_ms per session
- Error counter + trace ring buffer

Provider Flexibility

- Switch between 4 providers at runtime
- No cloud needed with Foundry Local
- Keyless Azure auth via Managed Identity

QUICK START

Up and Running in 3 Steps

01

Configure

Choose any of 4 providers — GitHub Models is free with no Azure subscription needed

```
cp .env.example .env
# Set MODEL_PROVIDER=github_models
# Set GITHUB_TOKEN=ghp_...
```

02

Start Backend

Script sets PYTHONPATH, creates .venv, installs deps, launches uvicorn with --reload

```
chmod +x start_backend.sh
./start_backend.sh
# FastAPI live at http://localhost:8000
```

03

Open Frontend

Upload sample_study_material.txt, ask questions, evaluate answers, watch traces live

```
cd frontend
python3 -m http.server 5500
# Open http://localhost:5500
```

Thank You

AI Tutor · Microsoft Agent Framework

RAG + ChromaDB | 4 MAF Providers | OpenTelemetry Traces | FastAPI Backend | Q&A Evaluation | VS Code AI Toolkit

Questions?

github.com · learn.microsoft.com/agent-framework · VS Code AI Toolkit